

values like Assembly, Manufacturing, Accounts rather than the integer values 0, 1, 2, etc. The following program illustrates the point I am trying to make.

```
#include <stdio.h>
#include <string.h>
void main()
{
    enum emp_dept
    {
        assembly, manufacturing, accounts, stores
    };
    struct employee
    {
        char name[30];
        int age;
        float bs;
        enum emp_dept department;
    };
    struct employee e;

    strcpy ( e.name, "Lothar Mattheus" );
    e.age = 28;
    e.bs = 5575.50;
    e.department = manufacturing;

    printf ( "\nName = %s", e.name );
    printf ( "\nAge = %d", e.age );
    printf ( "\nBasic salary = %f", e.bs );
    printf ( "\nDept = %d", e.department );

    if ( e.department == accounts )
        printf ( "\n%s is an accountant", e.name );
    else
        printf ( "\n%s is not an accountant", e.name );
}
```

And here is the output of the program...

```
Name = Lothar Mattheus
Age = 28
Basic salary = 5575.50
Dept = 1
Lothar Mattheus is not an accountant
```

Let us now dissect the program. We first defined the data type **enum emp\_dept** and specified the four possible values, namely, assembly, manufacturing, accounts and stores. Then we defined a variable **department** of the type **enum emp\_dept** in a structure. The structure **employee** has three other elements containing employee information.

The program first assigns values to the variables in the structure. The statement,

```
e.department = manufacturing;
```

assigns the value manufacturing to **e.department** variable. This is much more informative to anyone reading the program than a statement like,

```
e.department = 1;
```

The next part of the program shows an important weakness of using **enum** variables... there is no way to use the enumerated values directly in input/output functions like **printf()** and **scanf()**.

The **printf()** function is not smart enough to perform the translation; the department is printed out as 1 and not manufacturing. Of course, we can write a function to print the correct enumerated values, using a **switch** statement, but that would reduce the clarity of the program. Even with this limitation, however, there are many situations in which enumerated variables are god sent!